复制
> slave：执⾏复制命令[slaveof ip port]
> slave：保存master信息
> slave：定时任务发现新master，建⽴socket连接
> slave：ping请求，确认连接和master都ok，否则断开重连
> slave：权限验证
> ⾸次同步全量数据
> 持续复制：master持续地将写命令异步发给slave，保证数据⼀致
> ⼼跳检测：主从节点之间维护⼼跳和偏移量检测机制，保证通信正常
 集群
1.数据分区
将数据按分区规则划分到多个节点上，每个节点负责⼀个⼦集
> 哈希分区
> 顺序分区
2.节点通信
3.集群伸缩
4.故障转移
> 虚拟槽：设置16384个槽，采⽤CRC16(key)%16384算法将数据均匀散落在各个槽内，每个节点负责⼀部分槽
 优点：> 数据和节点解耦，⽅便节点扩容和收缩，客户端⽆需关⼼分区操作
 缺点：> 批量操作⽀持有限，只⽀持slot相同的key
 > 事务⽀持有限，只⽀持同⼀节点内事务
 > 不⽀持多数据库，只有⼀个数据库空间（单机redis最多⽀持16个数据库）
 > 不⽀持嵌套树状复制结构
> 节点取余：hash(key)%N，N为节点数量，计算出数据落哪个节点上（类似于HashMap的哈希寻址）
> ⼀致性哈希
P2P Gossip协议（节点彼此不断通信交换信息，⼀段时间后所有节点都会知道集群完整的信息）
> 每个节点单独开辟⼀个TCP通道，⽤于节点间通信
> 每个节点以固定的频率（10次/s）通过特定规则选择⼏个节点发送ping消息
> 当节点故障、新节点加⼊、主从⻆⾊变化等事件发⽣时，通过不断的ping/pong通信，⼀段时间后所有节点都会知道
> 节点A故障时，⽆法回复pong消息，超过timeout时间后发送节点会将其标记为「主观下线」
> 当集群中半数以上持有槽的主节点都认为A已下线时，触发「客观下线」流程
 > 向集群⼴播节点A的fail消息，所有节点标记A为客观下线并⽴刻⽣效
 > A节点的slaves开始故障转移
 > 所有slaves检查最后与master断线时间，符合⼀定资格才能参与master选举，并⼴播选举消息
 > 其他持有槽的主节点将票投给第⼀个请求投票的slave
 > 经过n轮选举，最终票数过半的的slave当选master，替换主节点
 > 取消数据复制，变为主节点，接管A负责的槽，⼴播⾃⼰的pong消息，通知集群
扩容：
> 启动新节点，任意节点中执⾏cluster meet，将新节点加⼊集群
> 经过⼀段时间的ping/pong之后，所有节点都已知晓新节点
> 迁移槽和数据
 > 确定迁移计划（哪些槽迁移给哪个新节点）
 > 迁移数据
 内存优化
1.内存消耗
 > key-value对象：存储键值对时需要创建key对象和value对象
 > 缓冲区：客户端、复制积压、AOF
 > 内存碎⽚：
 > ⼦进程内存消耗：持久化⼦进程
2.内存管理
 > 内存上限：maxmemory限制redis最⼤可⽤内存，可动态调整[config set maxmemory 2GB]
 > 内存回收策略
3.内存优化
 > 缩减key和value的⻓度：key越短越好
 > 优化字符串：redis实现了⾃⼰的字符串
 > 控制外层键的数量：将⼤量键分组存⼊hash结构中（适⽤于⼤量⼩对象的存储）
 > 共享对象池：内部维护[0,9999]的整数对象池（复⽤概率⼤，判断相等性的开销⼩）
删除过期键
> 惰性删除：等客户端读取时再删除（如果客户端⼀直不读取，会导致内存泄漏）
> 定时任务删除：运⾏频率10次/s，每次随机检查20个键，发现过期时删除
 如果过期率超过25%，循环执⾏回收逻辑

内存溢出控制策略（redis内存使⽤超过maxmemory时触发）
> 不删数据：写操作返回OOM，读操作正常响应（默认策略）
> 删数据：具体删哪些数据由具体策略决定
value
1.字符串：GZIP压缩json可节约60%的空间
2.java对象：> 精简业务对象，去掉不必要的属性，避免存储⽆效数据
 > 选择更⾼效的序列化⼯具
 事务
1.功能
 将多个指令打包，⼀次性按提交顺序执⾏，执⾏期间不响应其他客户端请求
2.命令
 > WATCH：乐观锁，监控指定key的改动情况（必须在multi之前watch）
 > MULTI：开启事务
 > 客户端提交命令，redis都存⼊queue
 > EXEC: 1).检查监控的key，如果有改动，告知客户端执⾏失败
 2).执⾏queue中的所有指令
 3).执⾏期间不会响应其他请求，如果exec前连接断了，清空queue
 > DISCARD：丢弃queue⾥的所有指令
3.特性
 > 不⽀持回滚：redis命令只会因为语法错误执⾏失败，redis作者认为此类编程错误应该在开发过程中避免，
 ⽽不是在⽣产环境中通过回滚处理（保持redis简洁快速）
 > 原⼦性范围：只保证命令的可执⾏性，如果queue⾥有错误的命令，整个事务都不会执⾏
 不保证命令执⾏后的正确性，如果执⾏命令时报错，继续执⾏下⼀条
 （这点和传统的关系型数据库事务不同）
4.替代⽅案-lua：客户端向redis发送lua脚本，redis原⼦性地执⾏后，返回结果
 缓存
1.收益
 > 加速读写，优化⽤户体验
 > 降低后端负载，例如⼀些复杂SQL计算，如果不加缓存，⽆法满⾜⾼并发量
2.成本
 > 数据不⼀致：取决于缓存更新策略
 > 代码维护成本：缓存层+存储层
3.使⽤场景
 > 开销⼤的复杂计算：复杂的DB联表操作、分组计算等，如果不加缓存，会给DB服务器带来巨⼤的负担
 > 加速请求响应：
4.更新策略
 > 超时剔除：先读缓存，过期时再读DB
 > 主动更新：DB更新后，刷新缓存
5.常⻅问题
 > 缓存穿透：⼤量并发访问缓存和DB都没有的数据，请求直接穿透到DB，缓存层形同虚设
 > 缓存击穿：⼤量并发访问DB有，但缓存已失效的数据，导致db压⼒激增
 > 缓存雪崩：⼤规模缓存失效or宕机，流量全部传到DB层，导致DB压⼒过⼤甚⾄宕机
 > ⽆底洞：集群⽔平扩容添加节点后，性能改善不⼤，甚⾄更糟糕
 > 热点key缓存重建
原因：业务数据被误删、恶意攻击
解决⽅案: 1.业务层校验，拦截不合法的请求，例如订单号为-1
 2.缓存空对象：设置缺省值，后续请求直接返回此值
 3.布隆过滤器
原因：⼤量缓存数据同⼀时间失效
解决⽅案: 1.过期时间分散开：过期时间+随机值
 2.热点数据分散到不同机器，构建⾼可⽤缓存集群
 3.限流：只允许核⼼业务请求访问DB
 4.服务降级，返回默认值or简单的提示信息
 布隆过滤器
1.结构：space efficient的概率型数据结构，⽤于判断⼀个元素是否在集合中
2.实现原理: > 分配⼀块内存做⼆进制数组，初始值全部为0
 > 添加key时，多个相互独⽴的hash函数寻址，将所有地址置1
 > 判断key是否存在：再次hash寻址，只要存在0，key⼀定不存在
3.使⽤场景: > 避免redis缓存穿透：将存在的key存⼊过滤器，不在过滤器中的key，不再查询缓存和DB
 > 过滤已爬过的⽹站、邮件⿊名单、已推荐的新闻等
解决⽅案: 1.过期时间分散开：过期时间+随机值
 2.DB访问限流：访问DB前，先获取互斥锁，只让⼀个线程通过
原因：节点增多，数据更分散，批量操作的⽹络操作次数增加
操作场景：批量获取n个字符串
解决思路：减少⽹络操作次数
解决⽅案： 1.分组+串⾏I/O：将key按节点分组，单线程为每组执⾏1次mget or pipeline
 2.分组+并⾏I/O：将key按节点分组，多线程为每组执⾏1次mget or pipeline
 3.使⽤hashtag将所有数据强制保存到⼀个节点上
分组原理：CRC16(key)找到slot，再找到负责该slot的节点
问题：热点key并发访问量⾮常⼤，但失效后重建缓存⾮常复杂，⼤量线程同时重建造成DB负载过⼤
解决⽅案: 1.限流，使⽤互斥锁，只允许⼀个线程重建，其他线程等待
 2.永不过期 + 异步重建（重建期间数据不⼀致）
> noeviction：不主动删数据，写操作返回OOM，读操作正常响应（默认策略）
> volatile-lru：删除「带过期时间的键空间」「最少使⽤的」key
> volatile-ttl：删除「带过期时间的键空间」「ttl较⼩的」key
> volatile-random：删除「带过期时间的键空间」「随机选择的」key
> allkeys-lru：删除「整个键空间」「最少使⽤的」key
> allkeys-random：删除「整个键空间」「随机选择的」key